

1. Grafana	2
1.1 Alerts, existing and required alerts for Monitoring 2.0 via Grafana	3
1.1.1 Grafana Alert Silences and Mute timings	11
1.1.2 Grafana CPU Alert for Linux servers	22
1.1.3 Integrating email as a Contact Point and adding it to Notification Policies in Grafana	24
1.2 Alloy Fleet Management	28
1.3 Beyla	29
1.3.1 Beyla VM Requirements Checkup List	32
1.4 Central Log monitoring (CLM)	33
1.4.1 CLM Helm Chart Overview (WIP)	35
1.4.2 How to write logs to CLM in PROD and DEV for Syslog	36
1.5 Grafana Cloud IPs allowlist	37
1.6 Grafana Cloud Probe Install	39
1.7 Grafana OTEL injector testing	42
1.8 Grafana ServiceNow integration	45
1.9 Using Grafana Azure Monitor to query Azure NSG Flow Logs	52

Grafana

Alerts, existing and required alerts for Monitoring 2.0 via Grafana

Default OS Alerts relevant to any server's base operating system, plus process down alert for Citrix defaults:

	Directory	Title	Query	type label	PROD Severity	UAT Severity PENDING IMPLEMENTATION	DEV Severity PENDING IMPLEMENTATION		Alert to SNOW PROD	Alert to SNOW DEV	Description	Pending	Clearing
1	Linux Server	Linux Process Offline Critical	group by (LOB, PRODUCT, Service_name, host, pattern, ENV, Assignment_Group, Assignment_Group_ID) (procstat_memory_usage{OS="Linux"} offset 1h unless on (LOB, PRODUCT, Service_name, host, pattern, ENV, Assignment_Group, Assignment_Group_ID))	\$labels.grafana_folder	critical	warning	warning			1	Linux Server process offline {{ \$labels.pattern }}	10m	10m
2	Linux Server	Ping Packet Percentage Loss Critical	label_replace(ping_percent_packet_loss{ENV="PROD", Assignment_Group="IT Ops ESM"}, "SEVERITY", "critical", "name", ".") or label_replace(ping_percent_packet_loss{ENV="PROD", Assignment_Group="IT Ops ESM"}, "SEVERITY", "warning", "name", ".")	\$labels.grafana_folder	critical	warning	warning		1	ADD	Linux Percentage Network packet loss is over 30%		
3	Linux Server	Ping Packet Percentage Loss Warning	label_replace(ping_percent_packet_loss{ENV="PROD", Assignment_Group="IT Ops ESM"}, "SEVERITY", "warning", "name", ".")	\$labels.grafana_folder	warning	warning	warning		1	1	Linux Percentage Network packet loss is over 20%		

4	Linux Server	Ping Latency Critical	label_replace(ping_averageresponse_ms{ENV="--PROD", Assignment_Group="IT Ops ESM"}, "SEVERITY", "critical", "name_") or label_replace(ping_averageresponse_ms{ENV="--PROD", Assignment_Group="IT Ops ESM"}, "SEVERITY", "warning", "name_")	\$Labels.grafana_folder	warning	no alert	no alert	1	1	Ping Latency Critical		
5	Linux Server	Linux CPU Utilization	sum by (host fs_type, Assignment_Group, Assignment_Group_ID) (round((100 - (cpu_usage_idle{cpu="cpu-total", OS="Linux"})) * 100, 2))	\$Labels.grafana_folder	critical	warning	warning	1	1	Linux High CPU Utilization		
6	Linux Server	Memory Usage Percentage Critical	sum by (host Assignment_Group, Assignment_Group_ID) (round((1 - (mem_available / mem_total{OS="Linux"})) * 100, 2))	\$Labels.grafana_folder	critical	warning	warning		1	Memory Usage Percentage Critical		
7	Linux Server	Disk Space Utilization Critical / Warning	(disk_used_percent{OS="Linux"} > 94.9 and disk_total{OS="Linux"} < 1000000000000) or (disk_used_percent{OS="Linux"} > 99 and disk_total{OS="Linux"} > 1000000000000) 90 to 94.9 = warning > 94.9 = critical for PROD, warning for non-PROD < 1 TB disk space = lower threshold > 1TB disk space = higher threshold	\$Labels.grafana_folder	critical	warning	warning		1	Linux Percentage Filesystem Used		
8	Linux Server	System Uptime (mins) Warn	system_uptime{}/60	\$Labels.grafana_folder	critical	warning	warning		1	System Uptime <5mins in Data Center: {{ \$Labels.Data_center }}, Host {{ \$Labels.host }}		

9	Linux Server	System Down Critical	absent(avg by (host)(system_uptime(OS="Linux") / 60))	\$labels.grafana_folder	critical	warning	warning			1	System Down / CPU stopped sending data for >5mins in Data Center: {{ \$labels.Data_center }}, Host {{ \$labels.host }}		
10	Linux Server	Network Latency Critical	sum by (host Data_center target_loc)(trace_route_response_time())	\$labels.grafana_folder	critical	warning	warning			1			
11	Windows Server	Windows CPU Utilization	sum by (host fstype Assignment_Group Assignment_Group_ID) (round((100 - cpu_usage_idle(cpu-total) OS="Windows"), 0))	\$labels.grafana_folder	{{ if and (gt \$values.A.Value 95.0) (eq \$labels.ENV "PROD") }} critical{{ else }} warning{{ end }}	{{ if and (gt \$values.A.Value 95.0) (eq \$labels.ENV "PROD") }} critical{{ else }} warning{{ end }}	{{ if and (gt \$values.A.Value 95.0) (eq \$labels.ENV "PROD") }} critical{{ else }} warning{{ end }}	1	1	Windows High CPU Utilization {{ printf "%.0f" \$values.A.Value }}% Windows High CPU Utilization			
12	Windows Server	Windows Memory Utilization	sum by (host Assignment_Group LOB_PRODUCT_Service_name Assignment_Group_ID ENV) (round((1 - mem_available / mem_total) OS="Windows" PRODUCT!="Originale Mortgagebot \ (Product\ Mortgagebot LOS \ (Product\))") * 100, 0))	\$labels.grafana_folder	{{ if and (gt \$values.A.Value 95.0) (eq \$labels.ENV "PROD") }} critical{{ else }} warning{{ end }}	{{ if and (gt \$values.A.Value 95.0) (eq \$labels.ENV "PROD") }} critical{{ else }} warning{{ end }}	{{ if and (gt \$values.A.Value 95.0) (eq \$labels.ENV "PROD") }} critical{{ else }} warning{{ end }}	1	1	Windows High Memory Utilization {{ printf "%.0f" \$values.A.Value }}%			

13	Windows Server	Windows Disk Utilization	sum by (hostfstype L OB PROD UCT ENV Service_name Assignment_Group Assignment_Group_ID device) (round((disk_used_percent{OS="Windows"fstype="NTFS"}), 2)) unless sum by (hostfstype L OB PROD UCT ENV Service_name Assignment_Group Assignment_Group_ID device) (round((disk_used_percent{PRODUCT="Mortgagebot LOS (Product)"service_name="IFL POD1-79"File Server", device="D."} or disk_used_percent{PRODUCT="Mortgagebot LOS (Product)"service_name="IFL POD8 - File Server", device="F."})))	\$Labels.grafana_folder	{{ if and (gt \$values.A.Value 95.0) (eq \$Labels.ENV "PROD") }}critical{{ else }}warning{{ end }}	{{ if and (gt \$values.A.Value 95.0) (eq \$Labels.ENV "PROD") }}critical{{ else }}warning{{ end }}	{{ if and (gt \$values.A.Value 95.0) (eq \$Labels.ENV "PROD") }}critical{{ else }}warning{{ end }}	1	1	Windows High Disk Utilization {{ \$Labels.device }} {{ printf "%.0f" \$values.A.Value }}%		
14	Windows Server	Telegraf Agent Down	count_over_time(system_uptime{OS="Windows"ENV="PROD"}, 10m)	\$Labels.grafana_folder	critical	warning	warning		1	Telegraf Agent Down		
15	Windows Server	Windows Service Down	(win_services_state{display_name =~ ".*", display_name =~ "Windows Update" display_name !~ "Grafana Agent" display_name !~ "SQL Server.*"} == 0)	\$Labels.grafana_folder	warning	no alert	no alert	1	1	Windows Service Down: {{ \$Labels.display_name }}		
16	Windows Server	Telegraf Agent Down Citrix	count_over_time(system_uptime{service_name =~ "Citrix.*", Assignment_Group =~ "IT Ops Intel"ENV="PROD DR ESS-PROD"}, 10m)	\$Labels.grafana_folder	critical	warning	warning	1	1	Telegraf Agent Down		
17	Windows Server	Windows Service Down Citrix	(win_services_state{display_name =~ ".*", service_name =~ "Citrix.*", Assignment_Group =~ "IT Ops Intel"} == 0)	\$Labels.grafana_folder	warning	no alert	no alert	1	1	Windows Service Down: {{ \$Labels.display_name }}		
18	Windows Server	Windows Service Down DHCP	(win_services_state{display_name =~ "DHCP.*"} == 0)	\$Labels.grafana_folder	warning	no alert	no alert	1	1	Windows Service Down: {{ \$Labels.display_name }}		

19	Fusion NetCapture	Linux Process Monitor - General	last_over_time(procstat_lookup_running PRODUCT="Fusion NetCapture" pattern ~"dgldgffle ss DG DG FF ESS" pattern~"dgff DgFileForwarder");30m))	\$Labels.grafana_folder	critical	no alert	no alert			1	{{ \$Labels.alertname }}		
	Directory	Title	Query	type label	Sev				PROD	DEV	Description		

DRAFT, this needs tweaking/consolidating

Alert Title	Class	Type Name	Tool Dashboard	URL to Windows Alert (if existent)	URL to Linux Alert (if existent)	description	Static Threshold	Priority	Severity	Query	Service Now	Knowledge Article Map	Automation Playbook Map
server down or unable to communicate	Base OS	Windows /Linux Default	Grafana		System Down Critical Alert	CPU data stops flowing after 3 consecutive polling periods			Critical	\$A = absent (avg by (host) (system_uptime) / 60)) trigger when \$A == 1 absent() function runs over the past 5 mins of data			
High CPU Utilization	Base OS				Windows High CPU Utilization	Threshold >90%	90		Warning	sum by (host, fstype, Assignment_Group) (round ((100 - cpu_usage_idle{cpu="cpu-total", OS="Windows"}), 2))			
High CPU Utilization	Base OS				Windows High CPU Utilization	Threshold >95%	95		Critical	sum by (host, fstype, Assignment_Group) (round ((100 - cpu_usage_idle{cpu="cpu-total", OS="Windows"}), 2))			
High Memory Utilization	Base OS				Windows High Memory Utilization	Threshold >90%	90		Warning	sum by (host, Assignment_Group) (round((1 - mem_available / mem_total {OS="Windows"}) * 100, 2))			
High Memory Utilization	Base OS				Windows High Memory Utilization	Threshold >95%	95		Critical	sum by (host, Assignment_Group) (round((1 - mem_available / mem_total {OS="Windows"}) * 100, 2))			

server disk space consumed too high	Base OS				Storage Critical Alert	threshold >90%	90		Warning	100 - ((node_filesystem_avail_bytes {mountpoint="/" , fstype!="rootfs"} * 100) / node_filesistem_size_bytes {mountpoint="/" , fstype!="rootfs"}))			
server disk space consumed too high	Base OS				Storage Critical Alert	threshold >95%	95		Critical	100 - ((node_filesystem_avail_bytes {mountpoint="/" , fstype!="rootfs"} * 100) / node_filesistem_size_bytes {mountpoint="/" , fstype!="rootfs"}))			
server uptime Indicates recent reboot	Base OS	Windows /Linux Default	Grafana		System Uptime (mins) Warn Alert	threshold <5mins			Warn	system_uptime{} / 60			
Telegraf process offline	Base OS	telegraf Linux /Windows process monitor	Grafana		Linux Process Offline Critical Alert	Process metrics fail to appear for 3 consecutive polls, fire a critical			Critical	procstat_lokup_running{} < 1			
Windows Service Down	Base OS	Windows Default				Service down, fire an alert			Critical				
Windows Process Down	Base OS	Windows Default				Process down, fire an alert			Critical				
NetCapture	Linux Process Monitor - General	Fusion NetCapture	Grafana										

Alerts for integrations above the base OS

IIS service not available	Windows	telegraf Windows iis monitor	Grafana										
traceroute failed between data centers	\$Data_center to \$target_loc failed \$details	telegraf Linux traceroute	Traceroute Between Datacenters - ESM - Dashboards - Grafana (provides.io)							sum by(host, target_loc, Data_center) (traceroute_response_time{Data_center="\$DATACENTER"}) when hop_num ==> max(hop_num) and target_ip != hop_host, then we can alert with the traceroute failed			
network latency between data centers is too high	\$Data_center to \$target_loc latency too high \$details	telegraf Linux traceroute	Traceroute Between Datacenters - ESM - Dashboards - Grafana (provides.io)		Network Latency Critical Alert	Latency >=900ms			Critical	sum by(host, Data_center, target_loc) (traceroute_response_time{})			
percent network packet drops between data centers is too high	\$Data_center to \$target_loc percent dropped packets too high \$details	telegraf Linux ping			Packet Loss Percentage Warn Alert	threshold 20% -30% packet loss			Warn	ping_percent_packet_loss{}			

[illegible]

Grafana Alert Silences and Mute timings

This page explains how to configure **Alert Silences** and **Mute Timings** in Grafana, and how to apply them to your alerting workflows. For additional details, see the official Grafana documentation on [Silences](#) and [Mute Timings](#).

1. Understanding Silences vs. Mute Timings

1.1 Silences

A **Silence** temporarily suppresses alert notifications for a defined time window (e.g., during maintenance).

Use when:

- You know the exact start and end time.
- Maintenance windows are one-time or manually scheduled.
- You want suppression for specific alert labels.

1.2 Mute Timings

A **Mute Timing** disables notifications based on a **recurring schedule**, such as:

- Specific days of the week
- Specific hours of the day
- Specific months or year ranges (e.g., first Monday of each quarter)

Use when:

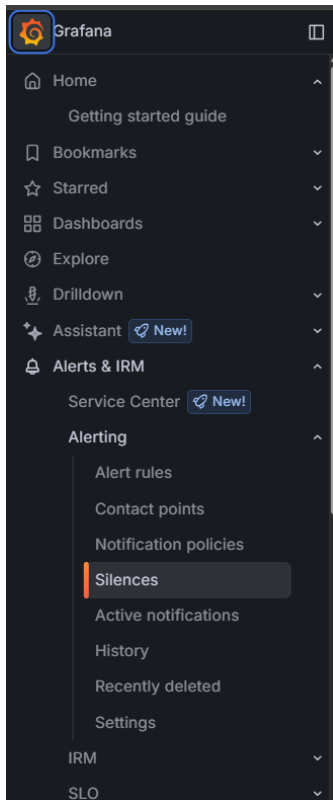
- You need repeated, predictable suppression (e.g., weekly maintenance).
- You want scheduled quiet periods embedded in notification policies.

2. Create Alert Silence

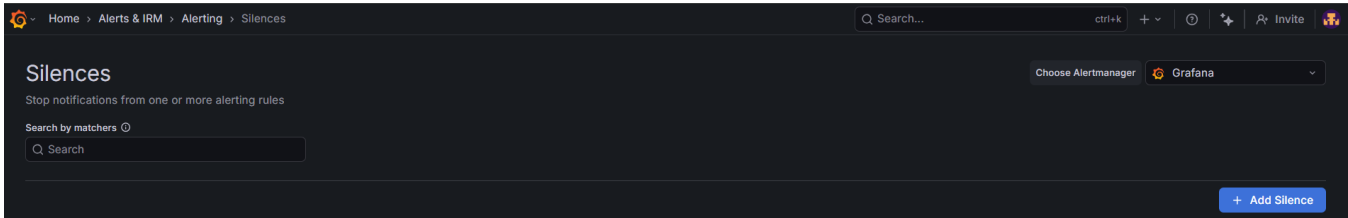
Silences are ideal for short-lived, one-off maintenance or troubleshooting periods.

Steps:

1. Click on Menu button then navigate to **Alerts & IRM** **Alerting** **Silences**



2. Click on Add Silence button



3. Configure the time range:

- Click on Silence start and end drop down
- Enter the Absolute time range
- Enter start (From) and end (To) for suppressing alerts
- Click Apply time range.

Note: Times default to **UTC**. Adjust if needed.

A screenshot of the 'Add silence' form in Grafana. The form has a title 'Add silence' and a subtitle 'Create silence'. It is divided into two main sections: 'Silence start and end' and 'Duration'. The 'Silence start and end' section has a text input field containing '2024-05-07 16:05:37 to 2024-05-07 16:30:37' with a dropdown arrow. The 'Duration' section has a text input field containing '25m'. Below these is the 'Absolute time range' section, which includes 'From' and 'To' labels, each followed by a text input field and a calendar icon. The 'From' field contains '2024-05-07 16:05:37' and the 'To' field contains '2024-05-07 16:30:37'. At the bottom of this section is a blue button labeled 'Apply time range'. At the very bottom of the form, there is a row with 'Browser Time Bahamas, EDT', a 'UTC-04:00' button, and a 'Change time settings' button.

3.2 Add Comment for reason/purpose of Silence, i.e. Ticket#, CTASK#, add comment below created date

A screenshot of the comment field in the Grafana Silence form. It shows a text area with the text 'created 2026-03-10 13:50' and 'CTASK0868418' on two lines. The text area is labeled 'Comment *' at the top left.

4. Add matching labels:

Configure the matching labels for alert to suppress this could be generic for a specific alert, host, PRODUCT, Service name. Examples:

Example	Effect
alertname = Windows Disk Utilization	Suppresses all alerts with this alertname
alertname = Windows Disk Utilization, instance = Server01	Suppresses alerts only for that host + alertname

Common label choice:

- **alertname**
- **host**
- **LOB**
- **PRODUCT**
- **Service_name**

Matching labels *

Label	Operator	Value	
host	=	VC01WSOMPR001	
Label	Operator	Value	
alertname	=	Windows Disk Utilizati	

5. Save silence to activate

Home > Alerts & IRM > Alerting > Silences > Silence alert rule

Silence alert rule

Configure silences to stop notifications from a particular alert rule

Silence start and end

Duration

2025-11-20 15:32:12 to 2025-11-20 15:57:12

25m

Refine affected alerts *

Label	Operator	Value	
host	=	VC01WSOMPR001	
alername	=	Windows Disk Utilizati	

+ Add matcher

Comment *

created 2025-11-20 15:32

Affected alert instances ⓘ

No firing alert instances found

Save silence

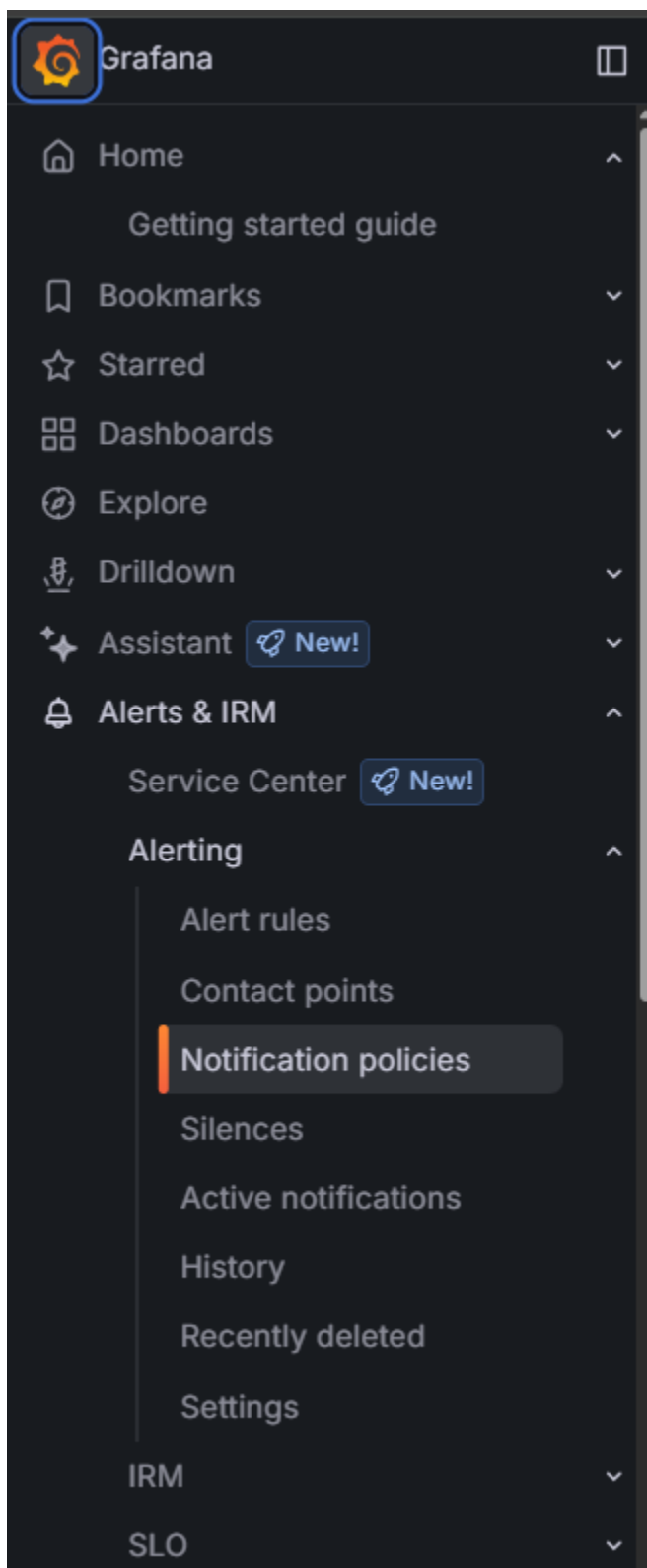
Cancel

3. Create Mute timing

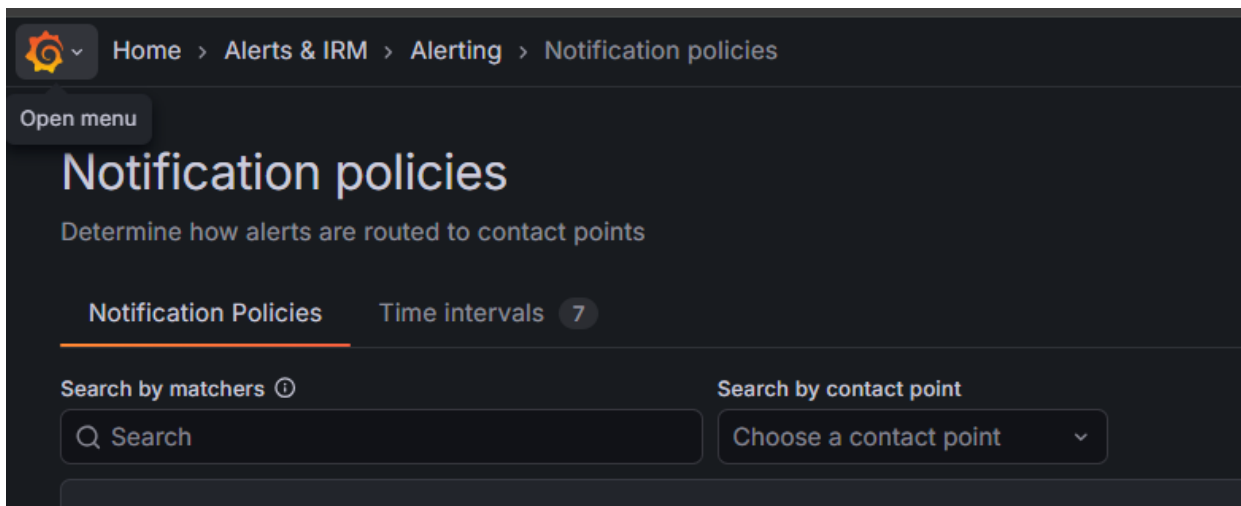
Mute timings are used to suppress alerts based on schedules and are applied through Notification Policies.

Steps:

1. Click on Menu button then navigate to **Alerts & IRM** **Alerting** **Notification policies**



2. Click on **Time Intervals** tab

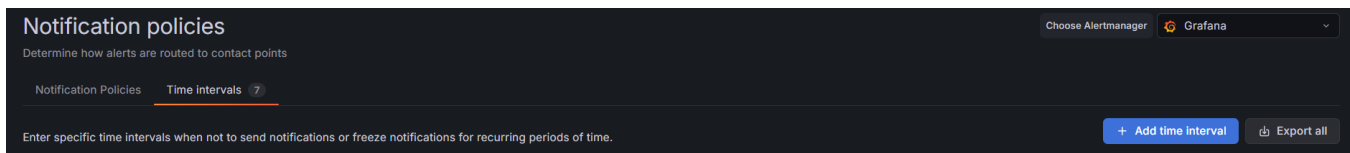


3. Click on Add interval in order to configure the time interval. Enter:

- Name
- Start time
- End time
- Optional schedule constraints:
 - Location (timezone)
 - Days of week
 - Days of month
 - Months
 - Years

You can configure multiple entries within a single Mute Timing if needed.

4. Click on the Save button



 Home > Alerts & IRM > Alerting > Notification policies > Add time interval

Add time interval

Name *
A unique name for the time interval

Time intervals

A time interval item is a definition for a moment in time. All fields are lists, and at least one list element must be satisfied to time interval, all fields must match. A time interval can contain multiple time interval items.

Time range
The time inclusive of the start and exclusive of the end time (in UTC if no location has been selected, otherwise local time)

Start time

HH:mm

End time

HH:mm

+ Add another time range

Location

Days of the week

Mon

Tue

Wed

Thu

Fri

Sat

Sun

Days of the month
The days of the month, 1-31, of a month. Negative values can be used to represent days which begin at the end of the month

Example: 1, 14;16, -1

Months
The months of the year in either numerical or the full calendar month

Example: 1;3, may;august, december

Years

Example: 2021-2022, 2030

Example mute timing for Mondays starting at 3pmEST and ending 3:30pmEST. Time zone is EST based on Location. Click on Save time interval to create interval.



Add time interval

Name *

A unique name for the time interval

ESM Scheduled Maintenance

Time intervals

A time interval item is a definition for a moment in time. All fields are lists, and at least one list must contain multiple time interval items.

Time range

The time inclusive of the start and exclusive of the end time (in UTC if no location has been selected, otherwise local time)

Start time

15:00



End time

15:30



+ Add another time range

Location



America/Toronto



Days of the week

Mon

Tue

Wed

Thu

Fri

Sat

Sun

Days of the month

The days of the month, 1:31, of a month. Negative values can be used to represent days which begin at the end of the month

Example: 1, 14:16, -1

Months

The months of the year in either numerical or the full calendar month

Example: 1:3, may:august, december

Years

Example: 2021:2022, 2030



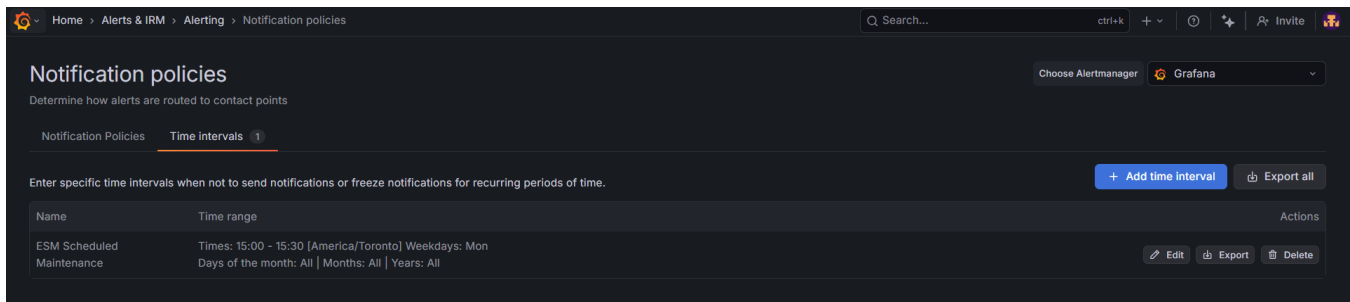
Remove time interval

+ Add another time interval item

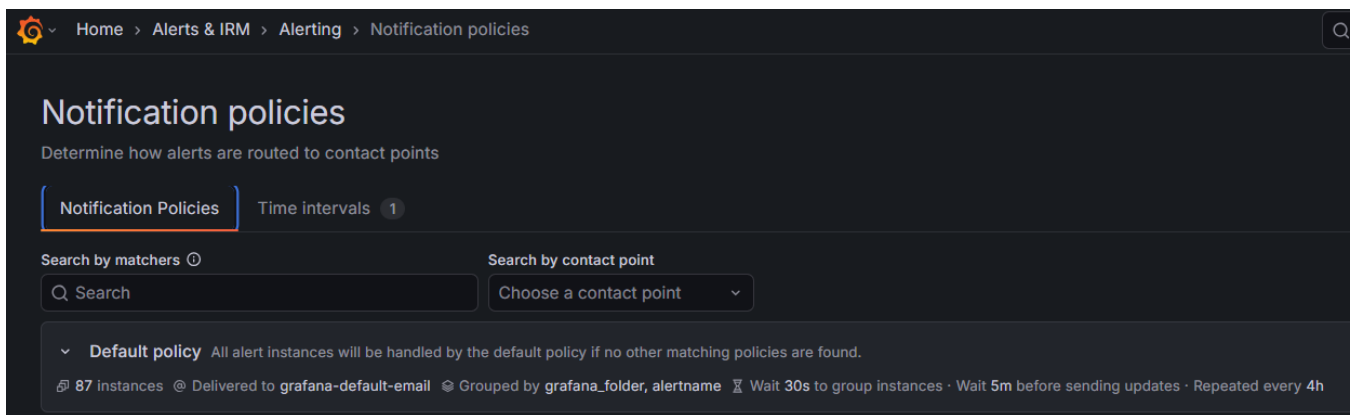
Save time interval

Cancel

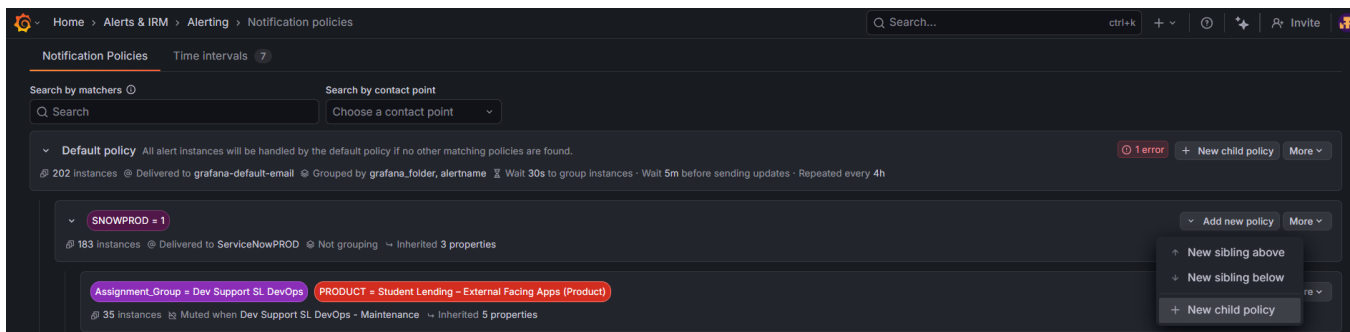
Once saved you will see your mute timing listed



5. Once a Mute Timing is created, you must attach it to a notification policy. Click on the **Notification policies** tab



6. Identify a parent policy (e.g. SNOWPROD=1, SNOWDEV=1). Click on **New child policy**



7. Configure matchers. Add labels that identify the alerts you want to mute, e.g.:

- host = server01
- PRODUCT = ProductA
- Service_name = SQL RemoteApp Server

8. Select the **Mute timings** that was previously created.

9. **Save** the policy

Add notification policy

Matching labels

Label

host

Operator

=

Value

VC01WSOMPR001

+ Add matcher

Contact point

Choose a contact point

Continue matching subsequent sibling nodes

Override grouping

Override general timings

Mute timings

Add mute timing to policy

ESM Scheduled Maintenance

Active timings

Add active timing to policy

Select time intervals...

Cancel

Save policy

Grafana supports complex combinations of date and time constraints.

Example: First Monday of March, June, September, December (12:00–24:00 UTC)

Field	Value
Start time	12:00
End time	24:00
Days of week	Mon
Months	3, 6, 9, 12 (or march, june, september, december)
Days of month	1:7 (first 7 days, ensuring the first Monday is included)

Location	(leave blank)
----------	---------------

Grafana CPU Alert for Linux servers

CPU alert rule for warning and critical threshold

1 Set an alert rule name

Rule name
Name for the alert rule.
CPU Utilization

2 Set a query and alert condition

Prometheus-Server now-15m to now Max data points 43200 Make this the alert condition

Kick start your query Explain

Metrics browser > `sum by(agent_hostname, assignment_group) (rate(node_cpu_seconds_total{mode!="idle"}[5m]))`

Options Legend: Auto Format: Time series Step: auto Type: Range

B Reduce

Function Last Input A Mode Strict

Make this the alert condition

C Math

Expression `$B > 0.20`

Alert condition

+ Add query + Add expression Preview

Prometheus query for CPU on Linux: `sum by(agent_hostname, assignment_group) (rate(node_cpu_seconds_total{mode!="idle"}[5m]))`

3 Alert evaluation behavior

Folder Linux Server Evaluation group (interval) CPU

Select a folder for your rule. Select a group to evaluate all rules in the same group over the same time interval.

Alert rules in the CPU group are evaluated every 1m. Evaluation group interval applies to every rule within a group. It overwrites intervals defined for existing alert rules.

Edit evaluation group

for 5m

Pause evaluation

Configure no data and error handling

Alert state if no data or all values are null
No Data

Alert state if execution error or timeout
Error

4 Add details for your alert rule

Write a summary to help you better manage your alerts. Click [here](#) for documentation on how to template annotations and labels.

Summary and annotations

Description CPU Usage greater than {{ \$values.B.Value | humanizePercentage }}

resource {{ \$labels.agent_hostname }}

Summary {{ \$values.B.labels.assignment_group }}

Choose Text

Add annotation Set dashboard and panel

5 Notifications

Grafana handles the notifications for alerts by assigning labels to alerts. These labels connect alerts to contact points and silence alert instances that have matching labels.

Custom Labels ⓘ

Labels	SNOWDEV	=	1	🗑️
severity		=	{{if gt \$values.B.V...	🗑️
type		=	{{ \$labels.grafana...	🗑️
Choose key		=	Choose value	🗑️
⊕ Add label				

Labels with conditions

```
severity={{if gt $values.B.Value 0.40 }}critical{{else}}warning{{end}}
```

```
type=(( $labels.grafana_folder }}
```

TIP to show all labels and values from alert rule

use Annotation

```
label (( print . }}
```

label	{{ print . }}	🗑️
-------	---------------	----

Integrating email as a Contact Point and adding it to Notification Policies in Grafana

This page explains how to configure **Email** as a notification contact point in Grafana and how to include it in your **Notification Policies**. For additional details, see the official Grafana documentation about [Contact points](#) or [Notification policies](#).

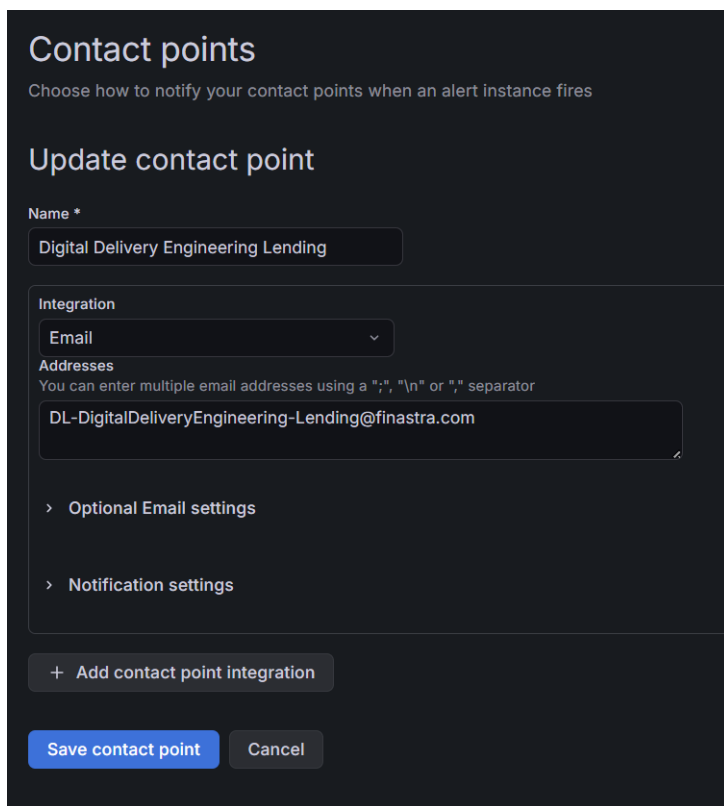
1. Adding Email as a Contact Point

To receive alert notifications via email, you must first configure an **Email contact point**.

Steps

1. Go to **Alerting Contact points**.
2. Click **Add contact point**.
3. Enter a name (e.g., *Email Notifications*).
4. Set **Type** to **Email**.
5. Add one or more recipient email addresses.
6. Click **Save contact point**.

Example UI:



The screenshot shows the 'Update contact point' interface in Grafana. At the top, it says 'Contact points' and 'Choose how to notify your contact points when an alert instance fires'. Below this is the 'Update contact point' section. It has a 'Name *' field with the value 'Digital Delivery Engineering Lending'. Under 'Integration', a dropdown menu shows 'Email'. The 'Addresses' section has a text input field with the email 'DL-DigitalDeliveryEngineering-Lending@finastra.com' and a note: 'You can enter multiple email addresses using a ";", "\n" or "," separator'. There are two expandable sections: 'Optional Email settings' and 'Notification settings'. At the bottom, there is a '+ Add contact point integration' button and two buttons: 'Save contact point' (in blue) and 'Cancel'.

Customize the email subject:

- Expand **Optional Email settings**
- Locate the **Subject** field

Enter a custom template using Go template syntax using labels, e.g.: `{{ .Status }}`, `{{ .CommonLabels.alertname }}`, `{{ .CommonLabels.host }}`, `{{ .CommonLabels.severity }}`, `{{ .CommonLabels.Service_name }}`, etc.

2. Adding the Email Contact Point to Notification Policies

Once the email contact point is created, you must add it to a **Notification Policy** so Grafana knows **when** and **for which alerts** to send emails.

2.1 Email-Only Routing (Single Contact Point)

Use this configuration when you want *all alerts* (or a specific subset) to be routed only to Email.

Steps

1. Navigate to Notification Policies
 - Go to **Alerting** **Notification policies**
2. Add a **Top-Level Policy**
 - Click "**New specific policy**" at the root level (same level as SNOWPROD=1, SNOWDEV=1, LCS-ASIS=1)
3. Configure the Policy
 - **Matching labels:** Add a unique routing label
 - **Label:** Custom label name (e.g., EMAIL_TEAM_X)
 - **Operator:** =
 - **Value:** 1
 - **Contact point:** Select your email contact point previously created
 - **Override grouping** (optional): Configure group_by labels
 - **Override general timings** (optional): Set group wait, group interval, repeat interval
 - Click **Save policy**
4. Add the **Routing Label** to Alert Rules
 - Open the alert rule(s) that should use this route
 - Scroll to the **Labels** section
 - Add the routing label: EMAIL_TEAM_X: 1
 - Save the alert rule

Add notification policy

Matching labels

Label	Operator	Value	
label	=	value	

+ Add matcher

Contact point

Digital Delivery Engineering Lending

Continue matching subsequent sibling nodes

Override grouping

Override general timings

Mute timings

Add mute timing to policy

Select time intervals...

Cancel

Save policy

Result

All alerts that match the policy's label rules will be routed **exclusively to Email**

2.2 Adding Email Notifications alongside ServiceNow (dual routing)

Use this approach when alerts should be sent to both ServiceNow AND email independently for specific hosts, products, or alert criteria.

When to Use

- Alerts already route to ServiceNow via SNOWPROD=1 label
- You want **some** alerts to also send emails (not all)
- You want to filter by specific hosts, products, services, or severity

To send alerts to both ServiceNow and email, you need **two sibling routes** under the existing SNOWPROD=1 or SNOWDEV=1 parent route:

1. **First route:** Sends to ServiceNow with "Continue matching" enabled
2. **Second route:** Sends to email contact point

Both routes use **identical matchers** to target the same alerts.

Steps

1. Navigate to Notification Policies

- Go to **Alerting Notification policies**
- Locate the parent route: {SNOWPROD=1} ServiceNowPROD or {SNOWDEV=1} ServiceNowDEV

2. Add First Nested Policy (ServiceNow Route)

- Click the "+" button or "New nested policy" on the {SNOWPROD=1} row
- Configure the matcher, e.g.:
 - Label: **host**, Operator: =~, Value: **w01p14002036001|w01p14006036001|w01p14001036001**
 - Label: **host**, Operator: =~, Value: **w01p1400.***
 - Label: **PRODUCT**, Operator: =, Value: **Mortgagebot LOS (Product)**
 - Label: **Service_name**, Operator: =, Value: **SQL RemoteApp Server**
- **Contact point:** Select ServiceNowPROD/ServiceNowDEV
- **Continue matching subsequent sibling nodes:** Check this box
- Click **Save policy**

3. Add Second Nested Policy (Email Route - Sibling to First)

- Click the "+" button again on the same {SNOWPROD=1} or {SNOWDEV=1} row (**not** inside the route you just created)
- Configure the matcher with **identical values** as the first route
- **Contact point:** Select your email contact point
- **Continue matching:** Leave unchecked
- Click **Save policy**

4. Verify Configuration

- You should see two sibling routes listed under {SNOWPROD=1} / {SNOWDEV=1}
- Both routes should have identical matchers
- First route should show "Continue matching: Yes"

The screenshot displays the 'Notification Policies' interface. At the top, the parent route 'SNOWPROD = 1' is shown with 387 instances and 'Delivered to ServiceNowPROD'. Below it, two nested policies are listed:

- Policy 1:** Assignment_Group = Dev Support SL DevOps, PRODUCT =~ Student Lending - Site24x7. It has 0 instances and is muted when Dev Support SL DevOps - Maintenance. It is highlighted with a red box.
- Policy 2:** ENV = PROD, PRODUCT =~ Fusion Originate ().*[Originate Consumer.*. It has 0 instances and is muted when Fusion Originate Consumer and Business Patch schedule. It is highlighted with a red box.

Below these, two more policies are shown, both with 0 instances and muted when specific maintenance modes are active:

- Policy 3:** host =~ lv-nc-uess1|sl-nc-pess1, service_name = BMOESS1. It is highlighted with a red box.
- Policy 4:** host =~ az-nc-pcm1|wf-nc-ucm1, service_name = WatcherBarclays. It is highlighted with a red box.

At the bottom, two more policies are listed, both with 1 instance and 'Delivered to ServiceNowPROD':

- Policy 5:** host =~ w01p14002036001|w01p14006036001|w01p14001036001|w01p14003036001|w01p14004036001|w01p14005036001|w01p14007036001|w01p14008036001|w01p14009036001|w01p1400b036001. It is highlighted with a red box.
- Policy 6:** host =~ w01p14002036001|w01p14006036001|w01p14001036001|w01p14003036001|w01p14004036001|w01p14005036001|w01p14007036001|w01p14008036001|w01p14009036001|w01p1400b036001. It is highlighted with a red box.

No Alert Rule Changes required:

- If alerts already route to ServiceNow, they have SNOWPROD=1 or SNOWDEV=1 label
- If alerts group by host or other labels, those labels are available for routing

Alloy Fleet Management

Configuration

Fleet Management can be activated in Alloy by adding the following to the config:

```
remotecfg {
  url = "https://fleet-management-prod-008.grafana.net"
  id = sys.env("GLOUD_FM_COLLECTOR_ID") // Unique ID for the alloy deployment - could be sys.env
("HOSTNAME")
  attributes = { "label" = "labelvalue", "label2" = "label2value" }
  basic_auth {
    username = "#####" // Instance id
    password = "#####" // API Token
  }
}
```

Requirements

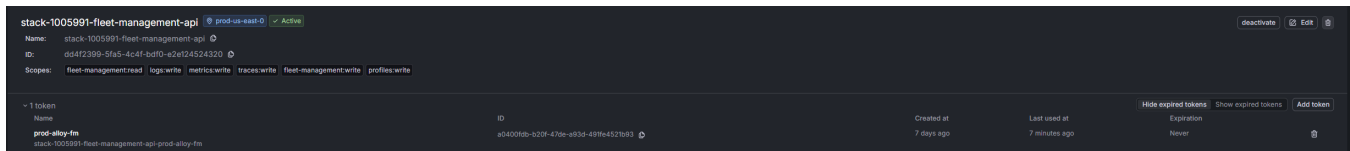
Clustering must be enabled if deployed in **Kubernetes**

Connection to the Fleet Management URL required - **external**

Components

- **URL** - standard URL address for Fleet Management - Checked from Grafana Cloud's Connections > Collector > Fleet Management > API
- **ID** - must be unique per instance - use env vars as in the example
- **attributes** - set label for labeling inside the Fleet Management screen - Can be LOB, Product, Service Tier, etc. - **Helps filtering later**
- **basic auth**:
 - **username** is the instance ID inside the API screen
 - **password** is one of the API tokens set up

API Token



The screenshot shows the 'stack-1005991-fleet-management-api' configuration page. It displays the API ID, scopes, and a table of API tokens. The table has columns for Name, ID, Created at, Last used at, Expiration, and actions (Hide expired tokens, Show expired tokens, Add token). One token is listed with the name 'prod-alloy-fm' and ID 'a04001db-920f-479e-a83d-49f64527693'.

Name	ID	Created at	Last used at	Expiration	
prod-alloy-fm	a04001db-920f-479e-a83d-49f64527693	7 days ago	7 minutes ago	Never	

API Tokens created are added to the **Cloud Access Policy** stack-xxxxxx-fleet-management-api and must have the scopes seen in the picture. Additional API Tokens are to be added to this Cloud Access Policy. Cloud Access Policies are in Administration > Users and Access > Cloud Access Policies

Management

Inside the **Inventory** tab of the Fleet Management page you can see the list of alloy deployments. Filters exist by multiple labels, **including the attributes set up previously**.

Here you can check various metrics and traces related to the alloy deployment (not host metrics).

Remote Configuration

It is also possible to apply config updates to alloy installations in Fleet Management, targeting is based **on predefined and custom attributes**, so they are very important in organizing the alloy installations into clusters.

Beyla

- **Key Feature:** app protocol observability - specialized for apps that are not instrumented internally, Beyla focuses on reading protocols used by an application and extracting data from them. Works with compiled binaries, no need to add SDKs to applications. Low overhead.
- Can be connected to **OpenTelemetry collectors**, **Grafana Cloud** or **Prometheus** instances. Can also send via Alloy to Grafana Cloud
- Focused on removing the need to restart applications, plug in and remove easily
- Uses **eBPF technology** which can run **sandboxed, user-defined programs** within the Linux Kernel
- Uses a **verifier** to make sure installation does not crash kernel and has protections against infinite loops
- Language agnostic approach
- Observability Signals:
 - RED metrics: Rate, Errors, Duration
 - Distributed traces: OpenTelemetry-compatible spans
 - Application metrics: HTTP, gRPC, SQL requests
 - Network metrics: TCP/UDP connections

Requirements

- **Linux with Kernel 5.8** or higher with **BTF enabled**, or Linux distributions running **RedHat Enterprise Linux 4.18** kernels build 348 and above as they have the required kernel backports. These include CentOS, AlmaLinux, and Oracle Linux. BTF became enabled by default on most Linux distributions with kernel 5.14 or higher. You can check if your kernel has BTF enabled by verifying if `/sys/kernel/btf/vmlinux` exists on your system. If you need to recompile your kernel to enable BTF, the configuration option `CONFIG_DEBUG_INFO_BTF=y` must be set.
- **eBPF enabled** on the host.
- For instrumenting Go programs, they must have been compiled with at least Go 1.17. We currently support Go applications built with a major Go version no earlier than 3 versions behind the current stable major release.
- Some level of elevated permissions to execute the instrumenter:
 - On host systems, running **Beyla requires sudo**.
 - For Kubernetes we have detailed configuration example on how to run with minimum required capabilities in the [examples/k8s/unprivileged.yaml](#) file.
 - For docker compose, you need to setup Beyla as privileged container or grant the **SYS_ADMIN** capability.

[Requirements Checkup List for VMs](#)

Docs: <<https://github.com/grafana/beyla?pg=oss-beyla&plcmt=hero-btn-2>>

Labels:

- Labels can be defined in the Yaml as:

```
attributes:

  environment: "prod"

  team: "payments"

  region: "us-east"
```

- Or via environment variables as:

```
export BEYLA_ATTRIBUTE_environment=prod

export BEYLA_ATTRIBUTE_team=payments
```

Config:

- Running: if using env vars make sure they pass to the process

```
sudo -E BEYLA_CONFIG_PATH=/home/opsapps1/beyla/beyla-config.yaml ./beyla

sudo -E BEYLA_CONFIG_PATH=/etc/beyla/config.yaml ./beyla
```

- Default paths (checked in order):

```
./beyla-config.yaml

./beyla-config.yaml

/etc/beyla/config.yaml

/etc/beyla/config.yaml
```

Basic Config Example

[Discovery modules](#)

[Grab Grafana OTLP Key here](#)

```
# specify jobs under discovery.instrument[] - name will be displayed in GC, ports specify which ports to monitor
discovery:
  instrument:
    - name: "app-test"
      open_ports: 4031,52311
    - name: "app2"
      open_ports: 553

otel_metrics_export:
  ttl: 5m
  endpoint: https://otlp-gateway-prod-us-east-0.grafana.net/otlp/v1/metrics
  protocol: http/protobuf
  features: ["network", "network_inter_zone"]
  buckets:
    duration_histogram: [0, 1, 2]
  histogram_aggregation: base2_exponential_bucket_histogram
  headers:
    Authorization: "Basic <Encoded Key>"

otel_traces_export:
  endpoint: https://otlp-gateway-prod-us-east-0.grafana.net/otlp
  protocol: http/protobuf
  instrumentations: ["http"]
  headers:
    Authorization: "Basic <Encoded Key>"

# Custom labels
attributes:
  environment: "prod"
  datacenter: "dc1"
```

Authorization

Authorization for sending Metrics and Traces requires an API Key. It is attached to a **Cloud Access Policy**, in Administration > Users and Access. The key is inside a policy with role set as **set:data-alloy-write**, called **stack-beyla**.

stack-beyla

prod-us-east-0

Active

Name:

stack-1005991-alloy

ID:

e4934442-19cb-4b74-b8f9-7fea7ccddbcf

Scopes:

set:alloy-data-write

1 token

Name	ID
test-beyla	d049bb40-1674-4f55-bf81-7dd24d91
stack-1005991-alloy-test-beyla	

The **safest way** of generating the Authorization header using the API key is to go to Connections > Add new connection > Beyla, then create a new secret there and copy the **OTEL_EXPORTER_OTLP_METRICS_HEADERS** and **OTEL_EXPORTER_OTLP_TRACES_HEADERS** and use those.

Environment Variables

Also, instead of adding the Authorization Keys in the config file, you may export the above values as **env vars**. Beyla will pick them up and use them for sending data.

```
export OTEL_EXPORTER_OTLP_TRACES_HEADERS="Authorization=Basic <Encoded Key>"
export OTEL_EXPORTER_OTLP_METRICS_HEADERS="Authorization=Basic <Encoded Key>"
```

Observability

In order to see the observability data of the instrumented apps in Grafana Cloud, go to Observability > Application and search the **Services** list.

Beyla VM Requirements Checkup List

```
uname -r #expected 5.8.x or 4.18.x-x.el8.x
ls /sys/kernel/btf/vmlinux #expected to exist
grep CONFIG_BPF_SYSCALL /boot/config-$(uname -r) #expected y
grep CONFIG_BPF_JIT /boot/config-$(uname -r) #expected y (optional, better performance)
cat /sys/kernel/security/lockdown 2>/dev/null || echo "File not found or lockdown not active" #expected none
```


Central Log monitoring (CLM)

CLM is the centralized log analytics engine focusing on logs from network devices to start, but branching out as budget allows to include infrastructure hardware, operating systems and application logs.

CLM is the first operationalized component in the general purpose, enterprise supported, Fusion Operate delivered kubernetes cluster.

CLM primarily contains Loki as the data lake for logs, Grafana as the visualization tool, but also contains Prometheus (wrapped in Mimir) for Metrics on such CIs as OS and application technology KPIs.

CLM is fed directly by network devices for their logs into a syslog component in the cluster, and is fed using the "monitoring 2.0" configuration management engine for logs and metrics from any other source.

Data in CLM is markedly different from regular syslog data, as the data contains enrichment labels that classify the data by business terms such as business unit, application, service, datacenter, environment.

1. Grafana:

- **Usage:** Grafana is like a smart dashboard for monitoring and visualizing data.
- **What It Does:** It helps you see important information (like system performance, website traffic, or sensor readings) in a clear and organized way. Think of it as your car dashboard showing speed, fuel level, and other stats.
- **Example:** Imagine you're tracking website visitors. Grafana can display real-time graphs, alerts, and summaries to help you understand how your site is doing.

2. Loki:

- **Usage:** Loki is like a super-powered search engine for logs.
- **What It Does:** When computers or apps write logs (like error messages or activity records), Loki collects and indexes them. It's like having a searchable diary for your systems.
- **Example:** If your app crashes, Loki lets you quickly find the relevant log entries to figure out what went wrong.

3. Mimir:

- **Usage:** Mimir is like a magical storage chest for long-term data.
- **What It Does:** It stores data (like metrics from servers or sensors) for a really long time. It's like putting your important files in a safe vault.
- **Example:** Mimir can hold years' worth of temperature readings, sales data, or any other valuable information.

PROD CLM Grafana console: <https://monitorclm.provides.io/login>

DEV CLM Grafana console: <https://monitor-dev.clm.provides.io/login>

Grafana SSO package

DEV CLM

Viewer

<https://myaccess.microsoft.com/@dh.com#/access-packages/68f8f9ed-85f5-4ebc-b8c2-1a0ed67196a8>

Admin

<https://myaccess.microsoft.com/@dh.com#/access-packages/4d0c7042-8c87-4735-9ce5-572bf20d3a63>

Editor

<https://myaccess.microsoft.com/@dh.com#/access-packages/54adf375-013a-42aa-8b9a-58dffaf9c777>

#####

PROD CLM

Viewer

<https://myaccess.microsoft.com/@dh.com#/access-packages/6ab0d228-d1e3-41f8-b105-363df6cbbf83>

Admin

<https://myaccess.microsoft.com/@dh.com#/access-packages/e4819c16-5c05-4e4b-a73b-a3f660943687>

Editor

<https://myaccess.microsoft.com/@dh.com#/access-packages/fba072f1-d192-49f9-8db4-83723109f465>

Documentation for Access Package: [Grafana SSO Access Package instructions](#)

CLM Github Repo: <https://github.com/Enterprise-Monitoring>

You need Azure DH tenant (firstname.lastname@dh.com) account

Azure AD groups for Github repo

Monitoring-Admin (SG_GitHub-FinastraEnterprise_Monitoring_Admin)

Monitoring-User (SG_GitHub-FinastraEnterprise_Monitoring_User)

Current Azure Resource Groups

CLM DEV

AZR-C01-ED-105-03 - Blob storage account (not used) (East US)

AZR-C01-ED-105-0G - Blob storage account (East US)

AZR-C01-ED-105-39 - AKS resources (East US)

AZR-C06-ED-105-38 - AKS replication (West US)

AZR-C06-ED-105-39 - AKS Overlay (West US)

CC01CRGPDV001 - Virtual Network (East US)

CC06CRGPDV001 - Virtual Network (West US)

CLM Dev cluster RG

~~mc_azr-c01-ed-105-39_clm k8s cluster dev_eastus (East US)~~ **DELETED**

MC_AZR-C06-ED-105-38_p0611053853a001_westus (West US)

MC_azr-c01-ed-105-39_p0111053953a001_eastus (East US)

MC_azr-c01-ed-105-39_p0111053953a002_eastus (East US)

MC_azr-c06-ed-105-39_p0111053953a003_westus (West US)

CLM PROD

~~AZR-C01-EP-105-0L - Old NFS storage account (East US)~~ **DELETED**

AZR-C01-EP-105-57 - AKS resources (East US)

AZR-C06-EP-105-67 - AKS replication RG (West US)

AZR-C01-EP-105-0L - Blob storage account (East US)

CC01CRGPPR001 - Virtual Network (East US)

CLM PROD cluster RG

mc_azr-c01-ep-105-57_p0121055753a001_eastus (East US)

MC_AZR-C06-EP-105-67_p0621056753a001_westus (West US)

CLM Helm Chart Overview (WIP)

CLM Product (ESM-GitOps-CLM-Product repo) we have the following structure:

- 1 app (one instance of prometheus, blackbox, loki, etc) with:
 - one parameters.yaml = defines the namespace, service account, and the helm repo and chart where our helm charts are re-packaged (esm-helm-charts repo)
 - one promotion.yaml = defines the order in which the app is deployed and on which environments it is deployed.
 - one values folder with one values file FOR EACH environment in promotion.yaml - name comes from promotion.yaml as follows: tier-class-region-environment. Each values file starts with:

```
values:
  mimic-distributed:
```

values:

chart-dependency: (the chart that was repackaged onto esm-helm-charts repo)

.....

The esm-helm-charts repo is structured as follows:

- One folder per chart per app with:
 - chart name, version, (copy paste most stuff), dependencies - dependencies link our chart to the public chart.
 - values file with default values (can be left empty, but file required)
- after creating the two files, run -> helm dependency build (in that folder) to fully repackage the chart
- you can create a templates folder and use it to define pvcs, scaling, etc

How to write logs to CLM in PROD and DEV for Syslog

1. Agent based configurations are 100% managed by IT OPS ESM, **no logs or metrics may flow into CLM without first being configured to do so by the Finastra Monitoring Configuration Engine.**
2. Network device logs MUST pipe logs to
 - a. DEV CLM: 10.199.192.190:514
 - b. PROD CLM: 10.199.180.64:514
 - c. These network device logs WILL NOT automatically be ingested into CLM. They will reach the syslog-ng tier, and sit at rest until such time as they're configured in the **Finastra Monitoring Configuration Engine.**
3. To be eligible for ingest into CLM, the source IP address must be configured in The Finastra Monitoring **Configuration** Engine

Agent configuration is a different path, these MUST be configured by the IT OPS ESM team. There are no times when agents may be configured outside of the IT OPS ESM team's engine.

CLM DEV:

Loki

10.199.192.190 port 514/1514 (syslog)

10.199.192.192 port 80/443 (grafana/telegraf agent)

Prometheus

10.199.192.192 port 80/443 (grafana/telegraf agent)

CLM PROD:

Loki

10.199.180.64 port 514/1514 (syslog)

10.199.180.68 port 80/443 (grafana/telegraf agent)

Prometheus

10.199.180.68 port 80/443 (grafana/telegraf agent)

Grafana Cloud IPs allowlist

If your network requires external services to be on an allowlist (formerly called a whitelist) to allow access, you can use the following lists to update your ACLs.

<https://grafana.com/docs/grafana-cloud/security-and-account-management/allow-list/>

Hosted Grafana

Hosted Grafana publishes similar lists of up-to-date IP addresses that can be used for allowlisting:

- JSON format: <https://grafana.com/api/hosted-grafana/source-ips>
- Text format: <https://grafana.com/api/hosted-grafana/source-ips.txt>
- DNS lookup: `src-ips.hosted-grafana.grafana.net`

Hosted Metrics

Hosted Metrics publishes similar lists of up-to-date IP addresses that can be used for allowlisting:

- JSON format: <https://grafana.com/api/hosted-metrics/source-ips>
- Text format: <https://grafana.com/api/hosted-metrics/source-ips.txt>
- DNS lookup: `src-ips.hosted-metrics.grafana.net`

Note

Instead of using these global lists, individual DNS lookup records are also available for each Hosted Metrics instance.

These records can be found in your Grafana Cloud stack details on the **Prometheus or Graphite instance details** page, in the **Allowing IPs for this cluster** section.

Hosted Logs

Hosted Logs publishes similar lists of up-to-date IP addresses that can be used for allowlisting:

- JSON format: <https://grafana.com/api/hosted-logs/source-ips>
- Text format: <https://grafana.com/api/hosted-logs/source-ips.txt>
- DNS lookup: `src-ips.hosted-logs.grafana.net`

Note

Instead of using these global lists, individual DNS lookup records are also available for each Hosted Logs instance.

These records can be found in your Grafana Cloud stack details on the **Loki instance details** page, in the **Allowing IPs for this cluster** section.

Hosted Traces

Hosted Traces publishes similar lists of up-to-date IP addresses that can be used for allow-listing:

- JSON format: <https://grafana.com/api/hosted-traces/source-ips>
- Text format: <https://grafana.com/api/hosted-traces/source-ips.txt>
- DNS lookup: `src-ips.hosted-traces.grafana.net`

Note

Instead of using these global lists, individual DNS lookup records are also available for each Hosted Traces instance.

These records can be found in your Grafana Cloud stack details on the **Tempo instance details** page, in the **Allowing IPs for this cluster** section.

Hosted Profiles

Hosted Profiles publishes similar lists of up-to-date IP addresses that can be used for allowlisting:

- JSON format: <https://grafana.com/api/hosted-profiles/source-ips>

- Text format: <https://grafana.com/api/hosted-profiles/source-ips.txt>
- DNS lookup: src-ips.hosted-profiles.grafana.net

Note

Instead of using these global lists, individual DNS lookup records are also available for each Hosted Profiles instance.

*These records can be found in your Grafana Cloud stack details on the **Pyroscope instance details** page, in the **Allowing IPs for this cluster** section.*

Grafana Cloud Probe Install

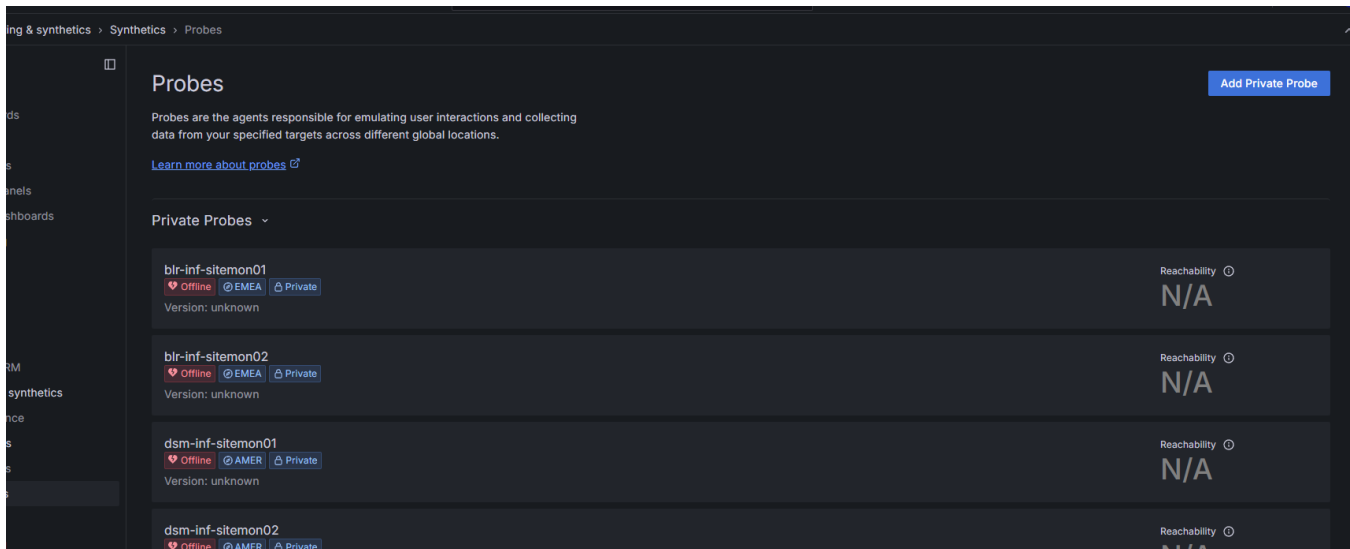
Perform connectivity test to ensure on premise monitoring poller can reach Grafana Cloud

Test connectivity

telnet synthetic-monitoring-grpc-us-east-0.grafana.net 443

Now you will need a token from the Grafana Cloud probe menu:

1) Top right add private probe



2) Set the private probe config:

Probe Name = Host Name

Location = approximate, map is only for visualization

Region = Region of the host

Check Disable Scripted checks

New private probe

This probe is private.

Private probes are operated by your organization and can only run your checks.

Probe Name *

Unique name for this probe.

lv-inf-sitemon01

Location information

Latitude *

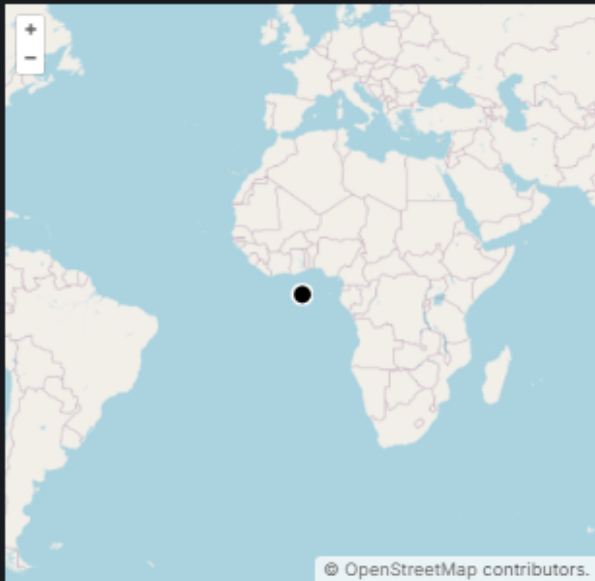
Latitude coordinates for this probe.

0

Longitude *

Longitude coordinates for this probe.

0



Click on the map to place the probe.

Region *

Region of this probe.

AMER

Labels

Custom labels to be included with collected metrics and logs. You can add up to 3. [Read more](#)

+ Add label

Capabilities



Disable scripted checks

Prevent probe from running k6 based scripted checks.

Backend address

The agent will need to connect to the instance of the Synthetics API that corresponds with the region of your stack. This is the backend address for your stack:

synthetic-monitoring-api-us-east-0.grafana.net

[Learn how to run a private probe](#)



Note

You must reconfigure any existing checks to use your new probe even if you selected all probes when initially creating the check.

Grafana Probe Install script

sudo wget -P /tmp https://cc01blrsglbessdv001.blob.core.windows.net/site24x7/packages/grafana_probe/GrafanProbeInstall.sh

Make it executable

```
sudo chmod +x /tmp/GrafanProbeInstall.sh
```

Run the script

```
/tmp/GrafanProbeInstall.sh
```

or

```
bash /tmp/GrafanProbeInstall.sh
```

Edit the Grafana Probe conf file

```
sudo nano /etc/synthetic-monitoring/synthetic-monitoring-agent.conf
```

Add the probe and copy the token to clipboard.

Paste the token generated when adding Probe in console and replace [Private Probe Agent Key]

i.e. SM_AGENT_API_TOKEN='xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'

```
# Enter API token retrieved from grafana.com here
SM_AGENT_API_TOKEN='[PRIVATE PROBE AGENT KEY]'

# Backend API server to connect to
API_SERVER='synthetic-monitoring-grpc-us-east-0.grafana.net:443'

# Don't use TLS with connections to GRPC API
API_INSECURE="false"

# listen address for /metrics and /debug/pprof
LISTEN_ADDRESS="127.0.0.1:4031"

# verbose logging
VERBOSE="false"

# debug output (enables verbose)
DEBUG="true"
```

then run:

Shell

```
sudo systemctl enable synthetic-monitoring-agent.service
```

```
sudo systemctl start synthetic-monitoring-agent.service
```

And check the logs for a READY state. If you don't get a READY state and instead encounter a TRANSIENT FAILURE or something similar, the logs need to be debugged, likely it can't connect to the cloud instance IP

Grafana OTEL injector testing

We will use sample java application provided by Grafana, the system will need docker installed as this is a docker application

Docker application file: <https://github.com/grafana/grafana-opentelemetry-java/blob/main/Dockerfile>

Grafana OTEL injector documentation: <https://github.com/jack-berg/injector-playground/tree/main>

Docker file tested and working with OTEL injector:

```
# Example production Dockerfile:
# - Spring petclinic: https://github.com/spring-projects/spring-petclinic
# - Alloy OR OpenTelemetry Collector

# Use renovate to keep the version up-to-date
# renovate: datasource=github-releases depName=grafana-opentelemetry-java packageName=grafana/grafana-opentelemetry-java
ARG GRAFANA_OPENTELEMETRY_VERSION=2.19.0@sha256:95eb14118e8a7ff517d7ce9f2ff288d793210fc8a3a0363c75f0d35ede3df40b
FROM us-docker.pkg.dev/grafanalabs-global/docker-grafana-opentelemetry-java-prod/grafana-opentelemetry-java:$GRAFANA_OPENTELEMETRY_VERSION AS agent

FROM springio/petclinic@sha256:fbb6f32dff6b4b05aa0bb4252a674a2084ecd26137793c88213385e62a6ea171

# 1. Install Alloy or OpenTelemetry Collector: https://grafana.com/docs/grafana-cloud/monitor-applications/collector

# 2. If Alloy or OpenTelemetry Collector is running on a different host, set the URL:
#ENV OTEL_EXPORTER_OTLP_ENDPOINT=http://10.49.37.74:43188

# 3. If Alloy or OpenTelemetry Collector does not offer gRPC, set the protocol to "http/protobuf":
#ENV OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf

# 4. Choose a **Service Name** to identify the service (e.g. `cart`)
#ENV OTEL_SERVICE_NAME=petclinic

# 4. Add attributes to filter data (recommended):
# - **deployment.environment**: Name of the deployment environment (`staging` or `production`)
# - **service.namespace**: A namespace to group similar services
#   (e.g. `shop` would create `shop/cart` in Application Observability)
# - **service.version**: The application version, to see if a new version has introduced a bug
# - **service.instance.id**: The unique instance, for example the Pod name (a UUID is generated by default)
#ENV OTEL_RESOURCE_ATTRIBUTES=deployment.environment=staging,service.namespace=shop,service.version=1

# Build and run the application:
# 5. docker build -t grafana_opentelemetry_java_demo .
# 6. docker run -p 8080:8080 grafana_opentelemetry_java_demo
# 7. Open <http://localhost:8080> in your browser
# 8. Click on "Error" in the top right corner to generate an error
# 9. Open Application Observability in Grafana Cloud:
#   - Click on the menu icon in the top left corner
#   - Open the "Observability" menu
#   - Click on "Application"
# Note: It might take up to five minutes for data to appear.

WORKDIR /app/

# If you want to let injector attach the agent, comment this out:
#COPY --from=agent --chown=cnb /javaagent.jar /app/javaagent.jar

#ENV JAVA_TOOL_OPTIONS=-javaagent:/app/javaagent.jar
```

Ensure Integration below for Alloy is applied in order to send metrics to Grafana Cloud

Grafana Alloy OTEL for Linux (typeID=91)

Grafana Alloy OTEL Profiles for Linux (typeID=93, please note only if profiling is required, also need to change Alloy service to run as root)

OTEL injector install

OTEL injector download <https://github.com/open-telemetry/opentelemetry-injector/releases>

OTEL injector install commands

- # Download opentel injector

```
/usr/bin/wget --no-check-certificate -O /tmp/opentelemetry-injector-0.0.3-1.x86_64.rpm 'https://downloads.clm.provides.io/linux-agents/opentelemetry-injector-0.0.3-1.x86_64.rpm?st=2025-08-14T08%3A46Z&se=2026-08-14T09%3A01Z&sp=rl&spr=https&sv=2022-11-02&ss=b&srt=co&sig=7O7x0CFLmZdOBKE8m55lnUH/M4vvWjfx/DmHTcZc28s%3D'
```

- # Install opentel injector

```
/usr/bin/rpm -ivh /tmp/opentelemetry-injector-0.0.3-1.x86_64.rpm
```

Files installed

ls /usr/lib/opentelemetry/

dotnet javaagent.jar js [libotelinject.so](#)

ls /etc/opentelemetry/

default_auto_instrumentation_env.conf otelinject.conf

- Add ENV variables below to **default_auto_instrumentation_env.conf** file (for Docker app IP should be 172.17.0.1)

```
OTEL_EXPORTER_OTLP_ENDPOINT=http://172.17.0.1:43188
```

```
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
```

```
OTEL_SERVICE_NAME=petclinic
```

```
OTEL_RESOURCE_ATTRIBUTES=deployment.environment=staging,service.namespace=shop,service.version=1
```

- Start docker application using the command below:

```
docker run -d -p 8080:8080 -v /usr/lib/opentelemetry:/usr/lib/opentelemetry:ro,Z -v /etc/opentelemetry:/etc/opentelemetry:ro,Z -e LD_PRELOAD=/usr/lib/opentelemetry/libotelinject.so grafana_opentelemetry_java_demo
```

Generate some traffic in the application and you should see the data in Grafana-->Observability-->Application

Commands to remove OTEL injector

- # Remove opentel injector

```
sudo rpm -e opentelemetry-injector
```

```
sudo rm -rf /usr/lib/opentelemetry
```

```
sudo rm -rf /etc/opentelemetry
```


Grafana ServiceNow integration

This document outlines the required configuration steps for integrating Grafana Alerting with ServiceNow (SNOW). It covers authentication, contact point configuration, required labels, notification policies, and ServiceNow field mappings.

1. ServiceNow Authentication

Request SN HTTP basic authentication credentials with SN team.

2. Grafana Contact Point Configuration

Create a new **Webhook** contact point in Grafana with the following settings:

ServiceNow PROD:

URL: https://finastra.service-now.com/api/sn_em_connector/em/inbound_event?source=grafana

ServiceNow DEV:

URL: https://finastradev.service-now.com/api/sn_em_connector/em/inbound_event?source=grafana

Integration : Webhook

HTTP method: POST

Input HTTP basic credentials (SN credentials stored in LastPass) You MUST NOT use the same authentication credentials as Enterprise monitoring. You MUST have unique credentials per instance of each tool.

Home

Alerting

Contact points

ServiceNowDev

Alerting

Alert rules

Contact points

Notification policies

Silences

Groups

Admin

ServiceNowDev

Edit the settings for a specific contact point

Alertmanager

Grafana

Update contact point

Name *

ServiceNowDev

Integration

Webhook

Test

Duplicate

Delete

URL

https://finastradev.service-now.com/api/sn_em_connector/em/inbound_event?

Optional Webhook settings

HTTP Method

POST

HTTP Basic Authentication - Username

grafana_snow

HTTP Basic Authentication - Password

Configured

Clear

Authorization Header - Scheme

Optionally provide a scheme for the Authorization Request Header. Default is Bearer.

Bearer

Authorization Header - Credentials

Credentials for the Authorization Request header. Only one of HTTP Basic Authentication or Authorization Request Header can be set.

Max Alerts

Max alerts to include in a notification. Remaining alerts in the same batch will be ignored above this number. 0 means no limit.

Title

Templated title of the message.

{{ template "default.title" . }}

Message

Custom message. You can use template variables.

{{ template "default.message" . }}

Notification settings

Disable resolved message

Disable the resolve message [OK] that is sent when alerting state returns to false

+ Add contact point integration

Save contact point

Cancel

3. Required Alert Labels

You MUST use labels to identify the service and CI impacted by the alert through service level mapping. Your label values MUST match the string values for the corresponding label names in Service Now. For example, assignment_group label may be called whatever you like, though we recommend calling it "assignment_group", when it arrives at Service Now, the value of the assignment group label must match a valid assignment group name. For example "IT OPS UNIX & Storage".

Business Attribute	Label Key	Description
Line of Business / Business Unit	LOB	Must match SNOW Business Unit value
Product / Business Application	PRODUCT	Must match SNOW Primary Business Service value
Service Name / Application Service Tier	Service_name	Must match role/service tier agreed during the onboarding process
Data Center	Data_center	Must match SNOW Location value

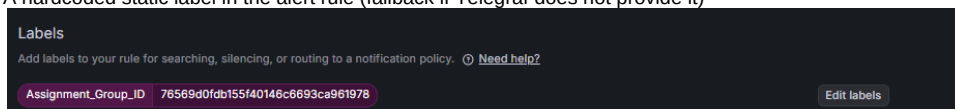
Environment	ENV	Must match SNOW Environment value
Assignment Group	Assignment_Group	Must match SNOW Support group value under the CI Team tab
Assignment Group ID	Assignment Group ID	Must match SNOW Assignment group sys_id value. Mandatory field!
Host Name / CI Name	Host	Must match SNOW Name value

You MUST include these labels in the description field so there is no question regarding the business impact.

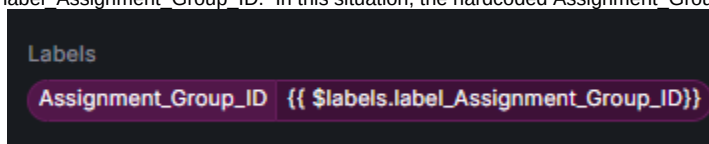
Mandatory Requirement: Assignment_Group_ID:

To ensure proper incident assignment in ServiceNow, the alert **must contain the exact ServiceNow sys_id** of the assignment group. This value is provided by:

- The Telegraf configuration ([global tags] section) for basic monitors or custom monitors contained in telegraf.conf file
- A hardcoded static label in the alert rule (fallback if Telegraf does not provide it)



- Important: Grafana alerts triggered by Synthetic Checks will not carry Assignment_Group_ID. Instead, they will expose the label under the synthetic prefix:
label Assignment_Group_ID. In this situation, the hardcoded Assignment_Group_ID label can be:



4. Notification Policies

Create a Notification Policy using the Contact Point created earlier:

- Ensure the policy matches alerts based on the environment label (SNOWDEV or SNOWPROD).
- Confirm that the correct contact point is applied for each environment.

Edit notification policy

×

Matching labels

Label	Operator	Value	
SNOWPROD	=	1	🗑️

+ Add matcher

Contact point

ServiceNowPROD

×

Continue matching subsequent sibling nodes

☐

Override grouping

☐

Override general timings

☐

Mute timings

Add mute timing to policy

Choose

Update policy

Cancel

Notification policies

Determine how alerts are routed to contact points

Choose Alertmanager

Grafana

Notification Policies

Mute Timings 0

Search by matchers

Search

Search by contact point

Choose

Default policy

All alert instances will be handled by the default policy if no other matching policies are found.

+ New nested policy

📧 Delivered to grafana-default-email

📁 Grouped by grafana_folder, alertname

SNOWPROD = 1

+ New nested policy

📧 Delivered to ServiceNowPROD

🔗 Inherited 1 property

5. Annotations required in Alert Rules:

The following **annotations** must be included to ensure proper incident rendering in ServiceNow:

- Description
- node
- resource
- metric
- assignment group

These ensure full context on impact, CI, and metrics.

4. Add annotations

Add annotations to provide more context in your alert notifications. ⓘ [Need help?](#)

Summary (optional)

Short summary of what happened and why.

Enter a summary...

Description (optional)

Description of what the alert rule does.

Windows High CPU Utilization

Runbook URL (optional)

Webpage where you keep your runbook for the alert.

https://

Custom annotation name and content

assignment group

{{ \$labels.Assignment_Group_ID }}



Custom annotation name and content

metric_name

{{printf "%.0f" \$values.A.Value }}



Custom annotation name and content

node

{{ \$labels.host }}



Custom annotation name and content

resource

{{ \$labels.host }}



6. Required Labels for DEV/PROD Environments

To route alerts to the correct SNOW instance, use the following labels:

Grafana DEV Alerts:

Label	Value
SNOWDEV	1
severity	(custom value)
type	(custom value)

Grafana PROD Alerts

Label	Value
SNOWPROD	1
severity	(custom value)
type	(custom value)

Example values for **severity** label:

- critical
- warning
- {{if eq \$labels.ENV "PROD" }}critical{{ else }}warning{{ end }} If the ENV label equals **PROD**, the alert severity will be **critical**. For any other environment, the severity will be **warning**.
- {{ if and (gt \$values.A.Value 95.0) (eq \$labels.ENV "PROD") }}critical{{ else }}warning{{ end }} if the value returned by the query is larger than **95%** and the ENV label equals **PROD**, the severity will be **critical**. Otherwise, the severity will be **warning**.

Example values for **type** label:

- {{ \$labels.grafana_folder }}

5

Notifications

Grafana handles the notifications for alerts by assigning labels to alerts. These labels connect alerts to contact points and silence alert instances that have matching labels.

Custom Labels ⓘ

Labels

SNOWDEV

=

1

severity

=

{{if gt \$values.B.V...

type

=

{{ \$labels.grafana...

Choose key

=

Choose value

⊕

Add label

7. ServiceNow mapping for Grafana Alerts

The following ServiceNow fields are concatenated into the **Incident Short Description**:

- Type
- Resource
- Description

Short description

Synthetic Monitors: [https \(https://3431250.app.netsuite.com/app/center/card.nl?sc=-46&whence=\)](https://3431250.app.netsuite.com/app/center/card.nl?sc=-46&whence=) - Grafana Monitoring detected low performance / latency for appli

The following fields are mapped to the **Incident Description**:

- Server Name
- Resource Name
- Additional Detail
- Message Key
- More Details

Description

Server Name : https

Resource Name : https://3431250.app.netsuite.com/app/center/card.nl?sc=-46&whence=

Additional Detail : {
"commonAnnotations": {},
"commonLabels": {
"SNOWDEV": "1",
"alertname": "\$INSTANCE Total Transaction Duration (s)",
"grafana_folder": "Synthetic Monitors",
"severity": "critical",
"type": "Synthetic Monitors"
},
"dashboardURL": "https://finfoprod105.grafana.net/d/ce9duues59gcga?orgId=1",
"endsAt": "0001-01-01T00:00:00Z",
"externalURL": "https://finfoprod105.grafana.net/",
"flattened.annotations.assignment_group": "48112a44db478700146c6693ca961970",
"flattened.annotations.description": "Grafana Monitoring detected low performance / latency for application Netsuite, with URL https://3431250.app.netsuite.com/app/center/card.nl?sc=-46&whence= from location NewYork",
"flattened.annotations.resource": "https://3431250.app.netsuite.com/app/center/card.nl?sc=-46&whence=",
"flattened.labels.SNOWDEV": "1",
"flattened.labels.alertname": "\$INSTANCE Total Transaction Duration (s)",
"flattened.labels.grafana_folder": "Synthetic Monitors",
"flattened.labels.instance": "https://3431250.app.netsuite.com/app/center/card.nl?sc=-46&whence=",
"flattened.labels.job": "Netsuite",
"flattened.labels.label_Assignment_Group_ID": "48112a44db478700146c6693ca961970",
"flattened.labels.label_Instance": "https://3431250.app.netsuite.com/app/center/card.nl?sc=-46&whence="

Server Name :

Input ▶ ... ▶ Node X

Resource Name :

Input ▶ ... ▶ Resource X

Additional Detail :

In... ▶ ... ▶ Additional informa... X

Message Key :

Input ▶ ... ▶ Message key X

More Details :

Input ▶ ... ▶ Short description X

Using Grafana Azure Monitor to query Azure NSG Flow Logs

NSG Flow Logs

1.) Query to Count Total Flows: This query counts the total number of flows, categorized by flow type: IntraVNet, InterVNet, S2S, P2S, AzurePublic, ExternalPublic, MaliciousFlow, Unknown Private, Unknown. It helps in understanding the volume of traffic.

```
AzureNetworkAnalytics_CL | where SubType_s == "FlowLog" | summarize count() by FlowType_s
```

2.) Query to Identify total number of Allowed and Denied Flows: This query categorizes flows into allowed and denied, helping to understand the effectiveness of NSG rules.

```
AzureNetworkAnalytics_CL | where SubType_s == "FlowLog" | extend AllowedOrDenied = iff(FlowStatus_s == 'A', 'Allowed', 'Denied') | summarize count() by AllowedOrDenied
```

3.) Query to Identify Traffic by Flow Direction: This query categorizes traffic into inbound and outbound, providing insights into the direction of traffic flow.

```
AzureNetworkAnalytics_CL | where SubType_s == "FlowLog" | extend FlowDirection = iff(FlowDirection_s == 'O', 'Outbound', 'Inbound') | summarize count() by FlowDirection
```

4.) Query to Identify Traffic by overall time intervals: This query provides a time-based count of traffic, useful for identifying traffic patterns and peak usage times.

```
AzureNetworkAnalytics_CL | where SubType_s == "FlowLog" | summarize count() by bin(TimeGenerated, 1h)
```

5.) Query to filter NSG flow logs in inbound or outbound directions: This query filters NSG flow logs, categorizes the flow direction, status, and protocol, and then projects specific fields for further analysis. This helps in understanding the traffic patterns, allowed or denied flows, and the protocols used in the network

```
AzureNetworkAnalytics_CL | where SubType_s == "FlowLog" | extend FlowDirection = iff(FlowDirection_s == 'O', 'Outbound', 'Inbound') | extend AllowedOrDenied = iff(FlowStatus_s == 'A', 'Allowed', 'Denied') | extend Protocol = case(L4Protocol_s == 'T', "TCP", L4Protocol_s == 'U', "UDP", L4Protocol_s) | project TimeGenerated, FlowDirection, AllowedOrDenied, SrcIP_s, DestIP_s, DestPort_d, Protocol, L7Protocol_s, NSGList_s, NSGRule_s
```

6.) Query to filter Mac Addresses in NSG flow logs: This query filters flow logs to include only those records with non-empty MAC addresses. This query then projects out the values

NOTE: Want distinct mac address values? at the end of the project use the distinct command

```
AzureNetworkAnalytics_CL | where SubType_s == "FlowLog" | where isnotempty(MACAddress_s) | project TimeGenerated, MACAddress_s, SrcIP_s, DestIP_s, DestPort_d, FlowDirection_s, FlowStatus_s, L4Protocol_s, NSGRule_s
```

7.) Query for searching within a specific NSG resource name: This query filters for logs which have that network security group name within it.

```
AzureNetworkAnalytics_CL | where NSGList_s endswith "eastus2"
```

8.) Query for searching for a destination port within a specific NSG: This query filters for logs which have that network security group that ends in eastus2 & has a destination port of 80

AzureNetworkAnalytics_CL| where NSGList_s endswith "eastus2"| where DestPort_d == 80

9.) Query for Top talkers: This query identifies the top source IPs for generating the most traffic.

AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(14d)| summarize TotalTraffic = sum(FlowCount_d) by SrcIP_s| top 10 by TotalTraffic desc

9.a) Query for Top Talkers to a specific IP Address in the NSG flow logs:

```
AzureNetworkAnalytics_CL
| where FlowStartTime_t > ago(3h)
| where DestIP_s in ("10.199.6.8", "10.199.6.9")
| summarize TotalTraffic = sum(FlowCount_d) by SrcIP_s
| top 10 by TotalTraffic desc
```

10.) Query for top destination ports: This query identifies which destination ports are receiving the most traffic.

AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(14d)| summarize TotalTraffic = sum(FlowCount_d) by DestPort_d| top 10 by TotalTraffic desc

11.) Query for top blocked source IP addresses: This query identifies the top source IPs blocked by NSG rules.

AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(18d) and FlowStatus_s == "D"| summarize BlockedTraffic = sum(FlowCount_d) by SrcIP_s| top 10 by BlockedTraffic desc

12.) Query for identifying unusual traffic spikes: Detect potential traffic spikes by comparing traffic volume over a period of time.

AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(17d)| summarize TotalTraffic = sum(FlowCount_d) by bin(FlowStartTime_t, 1h)| render timechart

13.) Query for identifying Top 5 Source-Destination Pairs Generating Most Traffic (Top talkers): This query helps you find the most active communication pairs between source and destination IPs, giving insights into the heaviest data paths in your network.

AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(18d)| summarize TotalTraffic = sum(FlowCount_d) by SrcIP_s, DestIP_s| top 5 by TotalTraffic desc| project SrcIP_s, DestIP_s, TotalTraffic

13.) Query for detecting spikes of denied traffic: This query can be used for tracking denied traffic over time to identify any sudden spikes that might indicate a potential issue or attack.

AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(15d) and FlowStatus_s == "D"| summarize DeniedTraffic = sum(FlowCount_d) by bin(FlowStartTime_t, 1h)| extend SpikeDetected = iff(DeniedTraffic > 1000, "Spike", "Normal")| project FlowStartTime_t, DeniedTraffic, SpikeDetected| order by FlowStartTime_t asc| render timechart

14.) Query for Identifying high traffic to specific ports: This query can be used breakdown of traffic by destination port, categorizing well-known ports and identifying the highest traffic destinations.

AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(18d)| summarize TotalTraffic = sum(FlowCount_d) by DestPort_d| extend PortCategory = case(DestPort_d == 80 or DestPort_d == 443, "HTTP/HTTPS", DestPort_d == 22, "SSH", DestPort_d == 3389, "RDP", "Other")| project DestPort_d, PortCategory, TotalTraffic| order by TotalTraffic desc

15.) Query for identifying malicious traffic in general: This query shows different values for malicious traffic in nsg flow logs.

```
AzureNetworkAnalytics_CL| where SubType_s == "FlowLog"| where FlowType_s == "MaliciousFlow"
```

16.) Digital Forensics Query for identifying suspicious traffic patterns: This query checks for blocked connections that may indicate brute force attacks or unauthorized access attempts, unusual traffic patterns.

NOTE: Run this kusto query all together not separate.

```
let blockedTraffic = AzureNetworkAnalytics_CL| where FlowStatus_s == "D" | summarize BlockedCount = count(), FirstBlockedTime = min(FlowStartTime_t), LastBlockedTime = max(FlowStartTime_t) by SrcIP_s, DestIP_s;let anomalousTraffic = AzureNetworkAnalytics_CL| where FlowStartTime_t > ago(7d) | summarize TotalTraffic = sum (FlowCount_d), ProtocolCount = dcount(Protocol_s) by SrcIP_s, DestIP_s| extend AnomalyScore = (TotalTraffic / 1000) + ProtocolCount| where AnomalyScore > 50 // Threshold for anomaly detection| project SrcIP_s, DestIP_s, TotalTraffic, AnomalyScore; // Include TotalTraffic herelet externalTraffic = AzureNetworkAnalytics_CL| where isnoteempty(DestIP_s) and DestIP_s !startswith "10." and DestIP_s !startswith "192.168." and DestIP_s !startswith "172." | summarize ExternalTrafficVolume = sum(FlowCount_d), MaxFlowCount = max(FlowCount_d) by SrcIP_s, DestIP_s;blockedTraffic| join kind=fullouter (anomalousTraffic) on SrcIP_s, DestIP_s| join kind=fullouter (externalTraffic) on SrcIP_s, DestIP_s| project SrcIP_s, DestIP_s, BlockedCount, FirstBlockedTime, LastBlockedTime, TotalTraffic, AnomalyScore, ExternalTrafficVolume, MaxFlowCount| where isnoteempty(AnomalyScore) or BlockedCount > 0 or ExternalTrafficVolume > 0| order by AnomalyScore desc, BlockedCount desc, ExternalTrafficVolume desc
```

17.) Query for identifying a spike in flows with nsg flow Logs. This can be useful for identifying billing spikes or flow spikes in an environment.

```
AzureNetworkAnalytics_CL| where SubType_s == "FlowLog"| extend FlowDirection = iff(FlowDirection_s == '0', 'Outbound', 'Inbound')| extend AllowedOrDenied = iff(FlowStatus_s == 'A', 'Allowed', 'Denied')| extend Protocol = case(L4Protocol_s == 'T', "TCP", L4Protocol_s == 'U', "UDP", L4Protocol_s)| project TimeGenerated, FlowDirection, AllowedOrDenied, SrcIP_s, DestPort_d, DestIP_s, DestPublicIPs_s, Protocol, L7Protocol_s, NSGList_s, NSGRule_s, NIC1_s, FlowCount_d, InboundPackets_d, OutboundPackets_d, InboundBytes_d, OutboundBytes_d
```

Public IP Details for NSG Flow Logs

1.) Query for analyzing Public IP details: This query is used for general analyzing of Public IP details for NSG flow logs.

```
AzureNetworkAnalyticsIPDetails_CL| where SubType_s == "FlowLog"
```

2.) Query for identifying malicious IPs: This query retrieves records of connections to known malicious IPs, including threat types and descriptions.

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "MaliciousFlow"| project FlowIntervalStartTime_t, FlowIntervalEndTime_t, IP_s, ThreatType_s, ThreatDescription_s, DNSDomain_s, Url_s, Port_s| order by FlowIntervalStartTime_t desc
```

3.) Query for identifying malicious IP addresses by country: This query summarizes the number of malicious IPs by country, helping to identify trends in attacks from specific locations.

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "MaliciousFlow"| summarize MaliciousCount = count() by Location_s| order by MaliciousCount desc
```

4.) Query for top threat types by malicious IP addresses: This query identifies the most common threat types associated with malicious IPs.

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "MaliciousFlow"| summarize ThreatCount = count() by ThreatType_s| order by ThreatCount desc
```

5.) Query for DNS Domains Associated with Malicious IPs: This query extracts DNS domains associated with malicious IPs

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "MaliciousFlow" and isnotempty(DNSDomain_s)| summarize DomainCount = count() by DNSDomain_s| order by DomainCount desc
```

6.) Query for flow log analysis by Public IPs: This query lists Azure Public IPs and their corresponding details in the logs.

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "AzurePublic"| project FlowIntervalStartTime_t, FlowIntervalEndTime_t, IP_s, PublicIPDetails_s, Location_s| order by FlowIntervalStartTime_t desc
```

7.) Query for identifying ExternalPublic traffic connections: This query identifies connections made to external public flows

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "ExternalPublic"| project FlowIntervalStartTime_t, FlowIntervalEndTime_t, IP_s, Location_s, PublicIPDetails_s| order by FlowIntervalStartTime_t desc
```

8.) Query for malicious flow duration: This query indicates how long a malicious flow was trying to attack an IP based component

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "MaliciousFlow" and FlowIntervalStartTime_t > ago(30d)| extend FlowDuration = datetime_diff('minute', FlowIntervalEndTime_t, FlowIntervalStartTime_t)| summarize AvgFlowDuration = avg(FlowDuration), MaxFlowDuration = max(FlowDuration), MinFlowDuration = min(FlowDuration) by IP_s| order by AvgFlowDuration desc
```

9.) Query for correlating malicious flows with external traffic: This query correlates malicious flows with external traffic, checking if any external connections were made to known malicious IPs.

NOTE: Run this kusto query all together not separate.

```
let externalTraffic = AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "ExternalPublic" and FlowIntervalStartTime_t > ago(19d)| project IP_s, FlowIntervalStartTime_t, FlowIntervalEndTime_t; let maliciousFlows = AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "MaliciousFlow" and FlowIntervalStartTime_t > ago(19d)| project IP_s, FlowIntervalStartTime_t, FlowIntervalEndTime_t; externalTraffic| join kind=inner (maliciousFlows) on IP_s| project IP_s, FlowIntervalStartTime_t, FlowIntervalEndTime_t| order by FlowIntervalStartTime_t desc
```

10.) Query for identifying repeated connections to the same malicious IP: This query identifies source IPs that make repeated connections to the same malicious IP over time.

```
AzureNetworkAnalyticsIPDetails_CL| where FlowType_s == "MaliciousFlow" and FlowIntervalStartTime_t > ago(30d)| summarize ConnectionCount = count() by IP_s, Port_s| where ConnectionCount > 10 // Adjust threshold as needed| project IP_s, Port_s, ConnectionCount| order by ConnectionCount desc
```